Treehouse tAVAX Merkl Audit Report

Sep 18, 2025

Table of Contents

Summary

This report has been prepared for Treehouse smart contract, to discover issues and vulnerabilities in the source code of their Smart Contract as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

Overview

Project Summary

Project Name	**Treehouse**
Codebase	**https://github.com/treehouse-gaia/tAVAX-protocol**
Commit	**5c74cf18c8f5a1059c69b1d16693a8afd8a1303f**
Language	**Solidity**

Audit Summary

Delivery Date	**Sep 18, 2025**
Audit Methodology	**Static Analysis, Manual Review**
Total Isssues	**2**

[WP-L1] When `toTreasury[i]` is `true` and there are duplicate tokens in the `tokens` array, the Strategy transfers its token balance to `TREASURY` .

Low

Issue Description

In this case, L87 `MERKL_DISTRIBUTOR.claim(users, tokens, amounts, proofs);` will only claim rewards once for each token type.

However, L89 - L93 will iterate multiple times for the same token, transferring `amounts[i] - claimed[i]` to `TREASURY` multiple times.

```
8   /// @title Action for claiming Merkl rewards
9   /**
10   * @dev Assumptions:
11   * - only recipient, or whitelisted operators, can claim rewards for recipient
12   * - length size of inputs ARE checked in merkl distributor
13   * - tokens and amounts are encoded in the proofs; if the token amounts are wrong
    code will revert
14   * - claimable amounts PASSED IN as assumed to be totals, the amount sent to
    treasury will be the available unclaimed portion in merkl
15   */
16  contract MerklClaim is ActionBase {
    @@ 17,19 @@
20
21    IDistributor constant MERKL_DISTRIBUTOR =
    IDistributor(0x3Ef3D8bA38EBe18DB133cEc108f4D14CE00Dd9Ae);
22    address constant TREASURY = 0x8AB0EB1314ffa9636B941E0d1c5805dec905B29a;
23
    @@ 24,42 @@
43
44    /// @inheritdoc ActionBase
45    function executeAction(
46      bytes calldata _callData,
47      uint8[] memory,
48      bytes32[] memory
49    ) public payable virtual override returns (bytes32) {
```

```
50      Params memory params = parseInputs(_callData);
51      (uint void, bytes memory logData) = _claim(
52        params.users,
53        params.tokens,
54        params.amounts,
55        params.proofs,
56        params.toTreasury
57      );
58      emit ActionEvent(NAME, logData);
59      return bytes32(void);
60    }
61
62    ////////////////////////// ACTION LOGIC //////////////////////////
63
64    /// @param users Recipient of tokens
65    /// @param tokens ERC20 claimed
66    /// @param amounts Amount of tokens that will be sent to the corresponding users
67    /// @param proofs Array of hashes bridging from a leaf `(hash of user | token |
    amount)` to the Merkle root
68    /// @param toTreasury Whether to send the claimed tokens to treasury
69    function _claim(
70      address[] memory users,
71      address[] memory tokens,
72      uint256[] memory amounts,
73      bytes32[][] memory proofs,
74      bool[] memory toTreasury
75    ) internal returns (uint, bytes memory) {
76      uint256[] memory claimed = new uint256[](tokens.length);
77
78      for (uint i; i < tokens.length; ++i) {
79        if (users[i] != address(this)) revert OnlyRecipient();
80
81        if (toTreasury[i]) {
82          (uint208 _claimed, , ) = MERKL_DISTRIBUTOR.claimed(address(this),
    tokens[i]);
83          claimed[i] = _claimed;
84        }
85      }
86
87      MERKL_DISTRIBUTOR.claim(users, tokens, amounts, proofs);
88
89      for (uint i; i < tokens.length; ++i) {
90        if (toTreasury[i]) {
```

```
91          IERC20(tokens[i]).safeTransfer(TREASURY, amounts[i] - claimed[i]);
92        }
93      }
94
95      bytes memory logData = abi.encode(tokens, amounts, toTreasury);
96      return (0, logData);
97    }
98
    @@ 99,113 @@
114 }
```

Recommendation

One approach is to use `MERKL_DISTRIBUTOR.claimWithRecipient()` with the `recipients` parameter (this is available on avalanche). Using `MERKL_DISTRIBUTOR.claimWithRecipient()` may help simplify the `MerklClaim` implementation.

Another approach could be to check the balance increment within the transfer loop. For duplicate token array items, the first item will transfer the tokens, and subsequent items will return 0 when querying the balance increment.

```
    @@ 64,68 @@
69    function _claim(
70      address[] memory users,
71      address[] memory tokens,
72      uint256[] memory amounts,
73      bytes32[][] memory proofs,
74      bool[] memory toTreasury
75    ) internal returns (uint, bytes memory) {
76      uint256[] memory preBalances = new uint256[](tokens.length);
77
78      for (uint i; i < tokens.length; ++i) {
79        if (users[i] != address(this)) revert OnlyRecipient();
80
81        if (toTreasury[i]) {
82          preBalances[i] =  tokens[i].balanceOf(address(this));
83        }
84      }
85
```

```
86      MERKL_DISTRIBUTOR.claim(users, tokens, amounts, proofs);
87
88      for (uint i; i < tokens.length; ++i) {
89        if (toTreasury[i]) {
90          uint256 toTransferAmount = tokens[i].balanceOf(address(this)) -
    preBalances[i];
91          if (toTransferAmount > 0) {
92            IERC20(tokens[i]).safeTransfer(TREASURY, toTransferAmount);
93          }
94        }
95      }
96
97      bytes memory logData = abi.encode(tokens, amounts, toTreasury);
98      return (0, logData);
99    }
```

Status

✓ Fixed

[WP-N2] `tAVAX` should reference the code on the Avalanche chain instead of `etherscan` .

Issue Description

The Merkl Distributor code on Ethereum is different from the one on Avalanche.

https://github.com/treehouse-gaia/tAVAX-protocol/blob/fde4543e0c873320c5c590cdfd93ec9644dede96/contracts/interfaces/merkl/IDistributor.sol#L1-L29

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity =0.8.24;
3
4  // https://etherscan.io/address/0x3Ef3D8bA38EBe18DB133cEc108f4D14CE00Dd9Ae#readProxyContract
5  interface IDistributor {
6    /// @notice Claims rewards for a given set of users
7    /// @dev Anyone may call this function for anyone else, funds go to destination regardless, it's just a question of
8    /// who provides the proof and pays the gas: `msg.sender` is used only for addresses that require a trusted operator
9    /// @param users Recipient of tokens
10   /// @param tokens ERC20 claimed
11   /// @param amounts Amount of tokens that will be sent to the corresponding users
12   /// @param proofs Array of hashes bridging from a leaf `(hash of user | token | amount)` to the Merkle root
13   function claim(
14     address[] calldata users,
15     address[] calldata tokens,
16     uint256[] calldata amounts,
17     bytes32[][] calldata proofs
18   ) external;
19
20   /// @notice Toggles whitelisting for a given user and a given operator
21   function toggleOperator(address user, address operator) external;
22
23   function operators(address user, address operator) external view returns (bool isAuthorized);
24
25   function claimed(
```

```
26      address user,
27      address token
28    ) external view returns (uint208 amount, uint48 timestamp, bytes32 merkleRoot);
29  }
```

Status

✓ Fixed

Appendix

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by WatchPug; however, WatchPug does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication.

Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Smart Contract technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.